Secure Multi-Tier Web Application Architecture in Cloud

1. Introduction

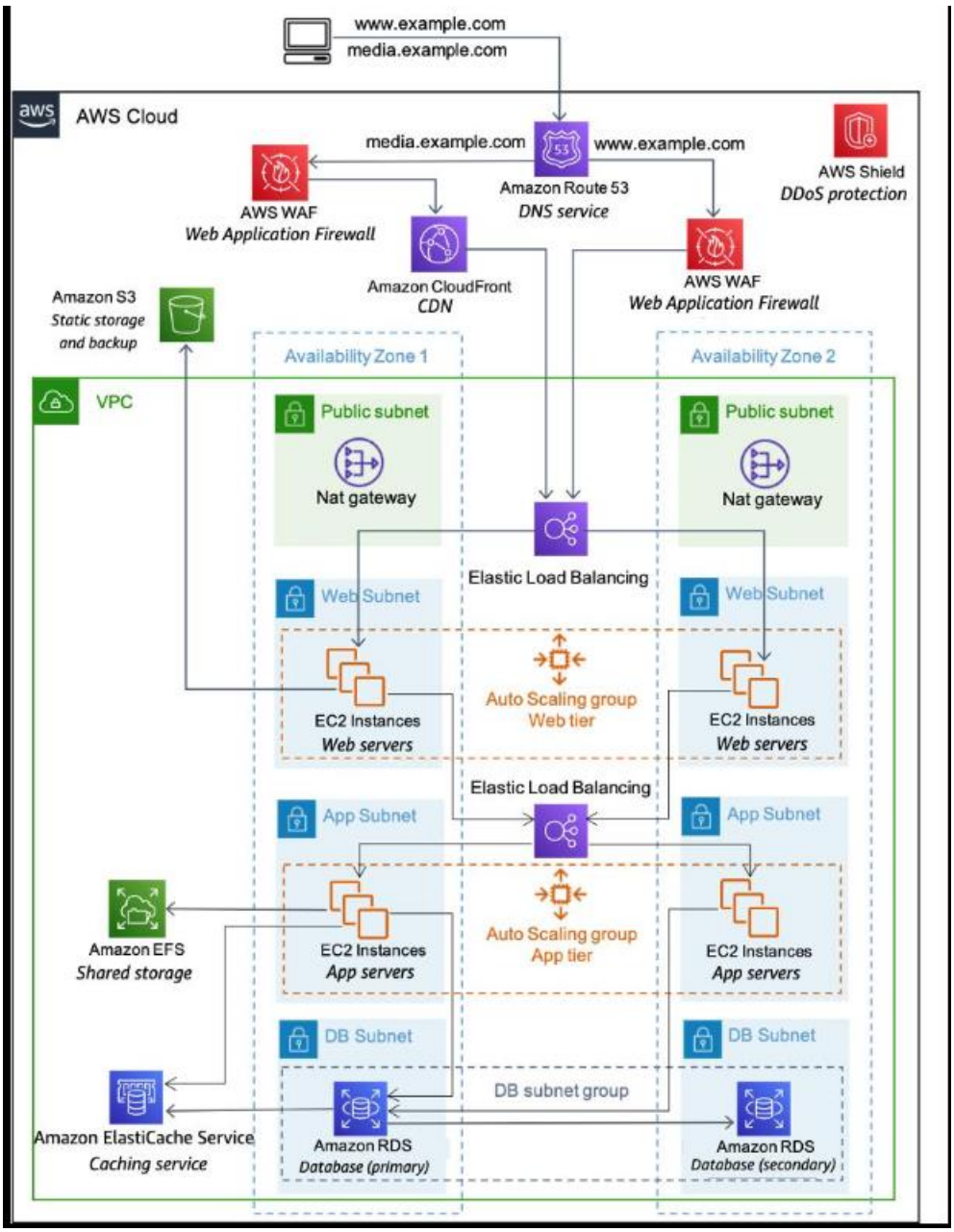
This project focuses on designing and analyzing a secure multi-tier web application architecture in a cloud environment. The goal was to understand secure communication between public-facing services and internal backend systems

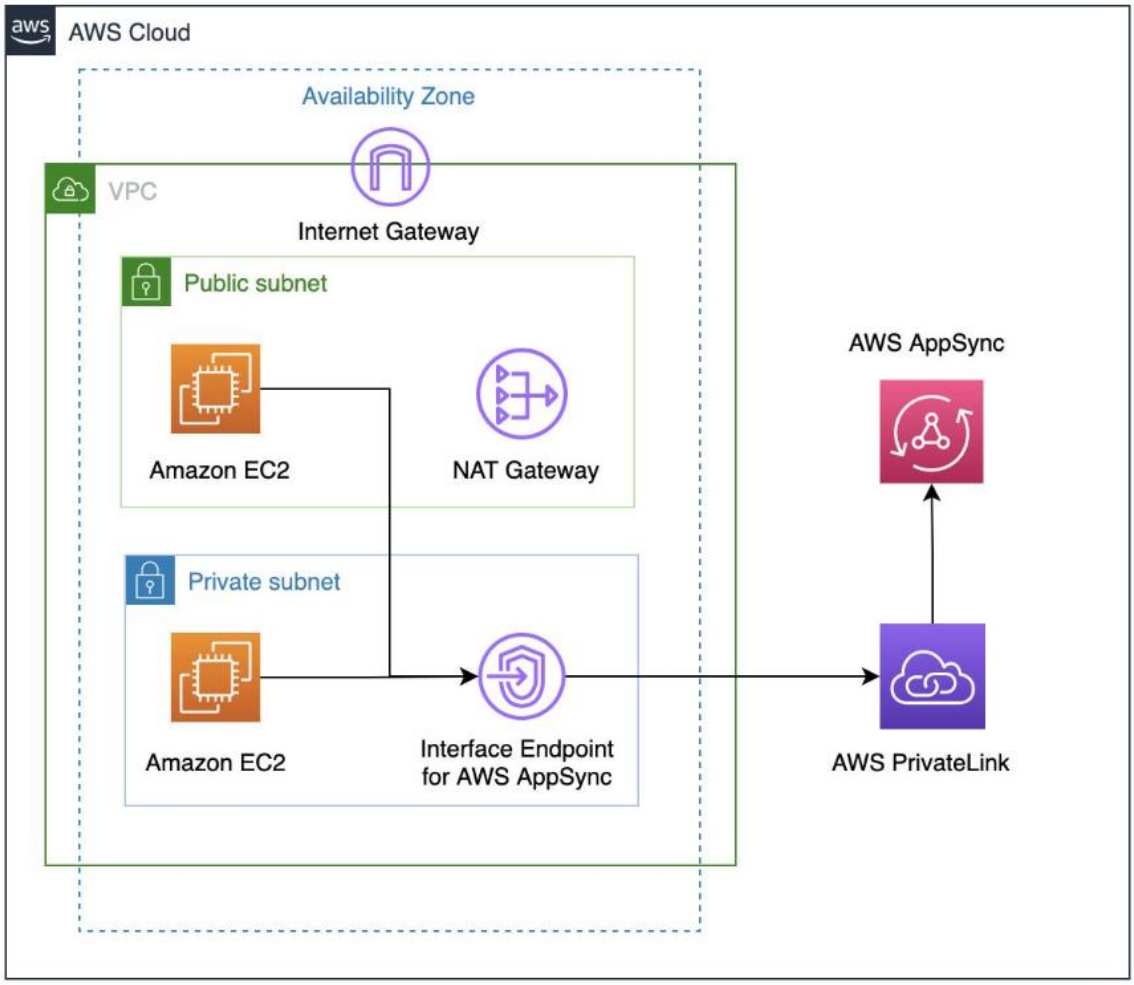
2. Architecture

The architecture includes:

- Public web application (internet-facing)
- Backend services hosted on private instances
- VPC with public and private subnets
- Security groups controlling access
- TLS-based communication between services

Architecture diagrams on the next page:


www.example.com
media.example.com
AWS Cloud
media.example.com
www.example.com
Amazon Route 53
DNS service
AWS Shield
DDoS protection
AWS WAF
Web Application Firewall
Amazon CloudFront
CDN
AWS WAF
Web Application Firewall
Amazon S3
Static storage
and backup
Availability Zone 1
Availability Zone 2
VPC
Public subnet
Nat gateway
Public subnet
Nat gateway
Elastic Load Balancing
Web Subnet
Web Subnet
EC2 Instances
Web servers
Auto Scaling group
Web tier
EC2 Instances
Web servers
Elastic Load Balancing
App Subnet
App Subnet
Amazon EFS
Shared storage
EC2 Instances
App servers
Auto Scaling group
App tier
EC2 Instances
App servers
DB Subnet
DB Subnet
DB subnet group
Amazon ElastiCache Service
Caching service
Amazon RDS
Database (primary)
Amazon RDS
Database (secondary)



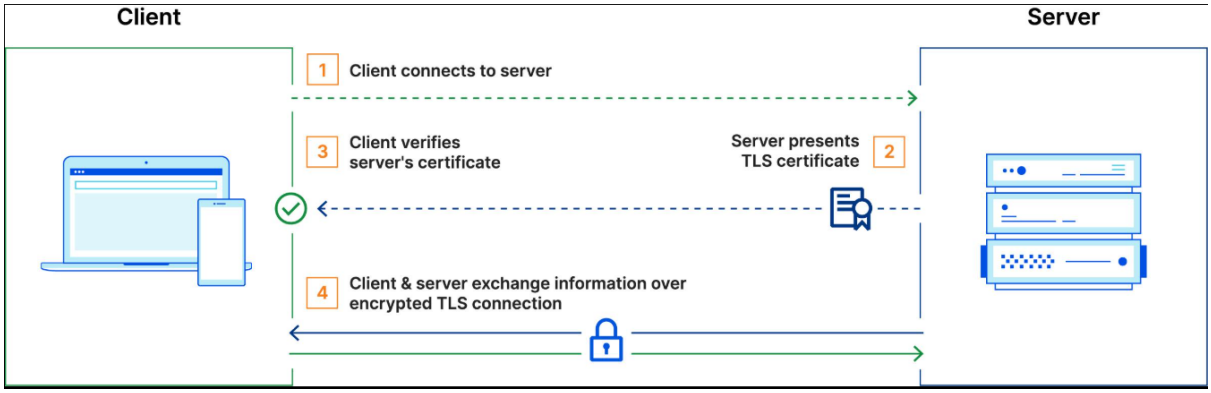
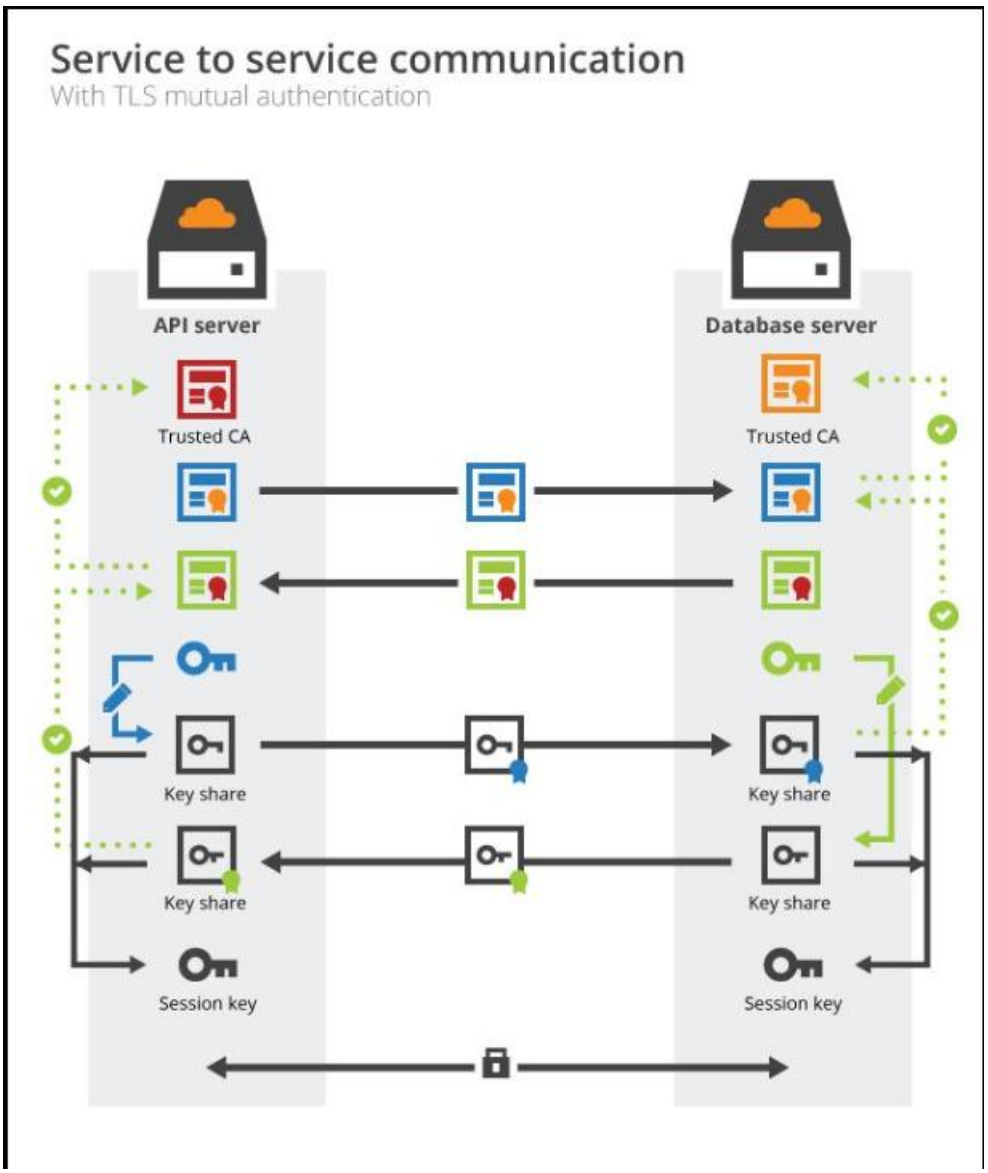


Fig: TLS handshake overview


Service to service communication
With TLS mutual authentication
API server
Database server
Trusted CA
Trusted CA
Key share
Key share
Key share
Key share
Session key
Session key

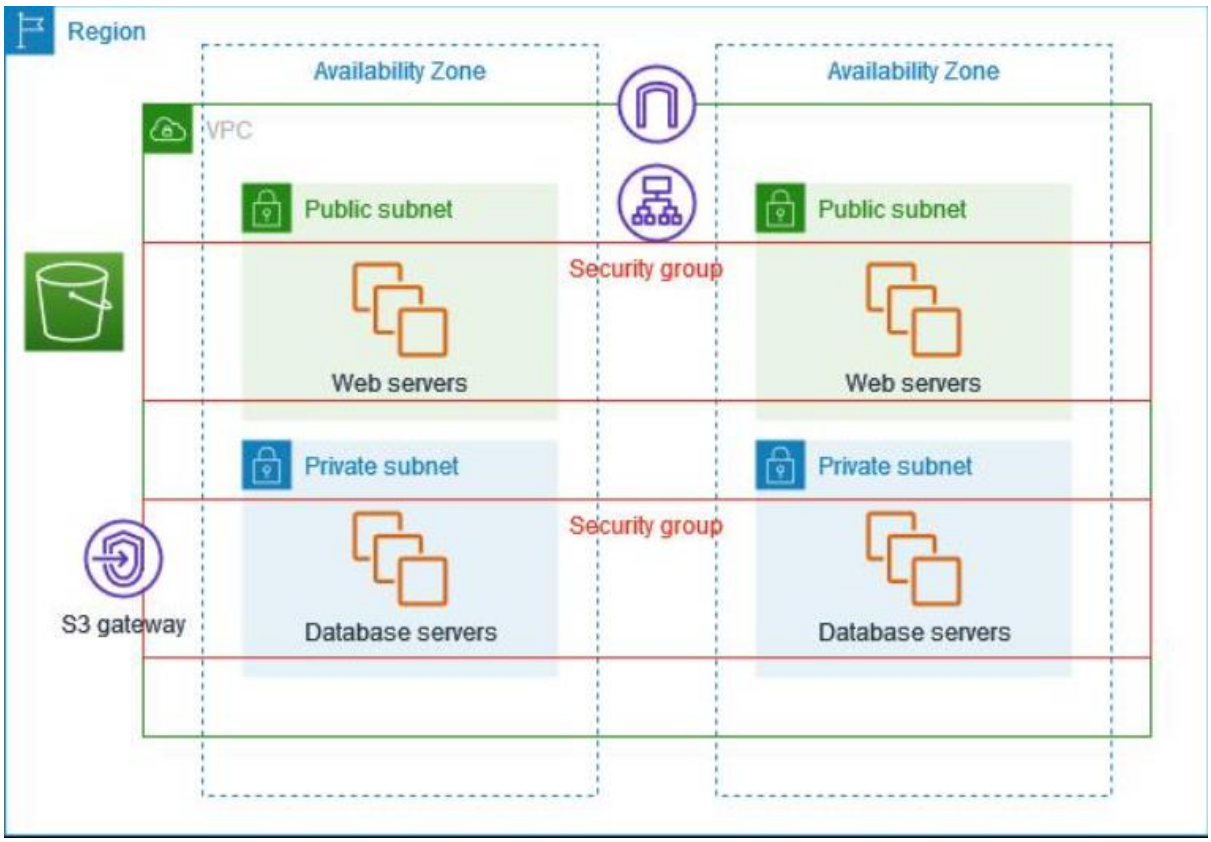
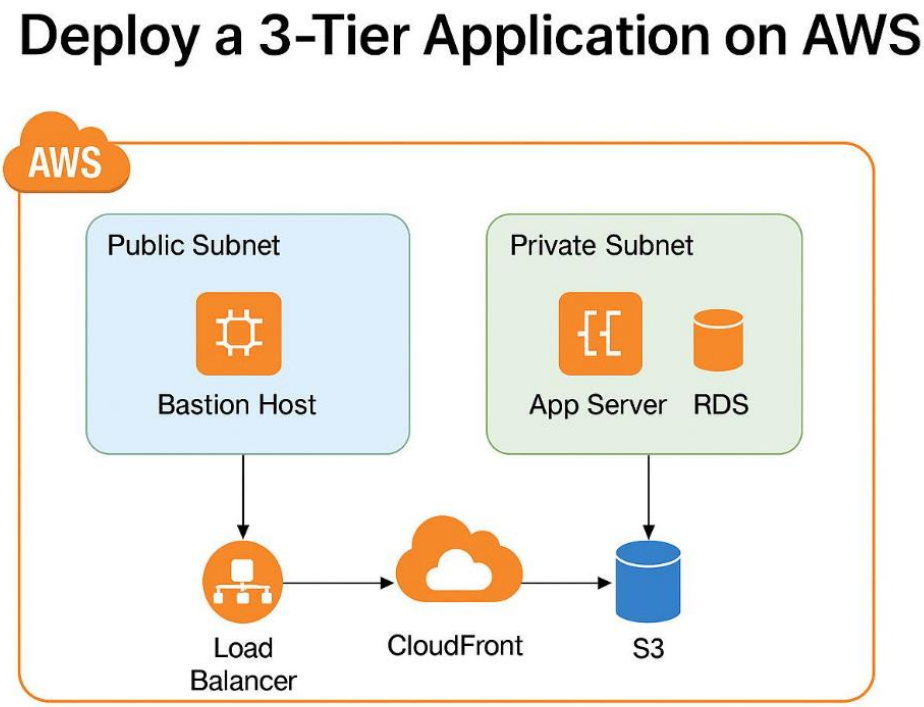


Fig: Overall workflow



3. Implementation & Testing

The following tasks were performed as part of the testing :

- Deployment of public and private web applications
- Configuration of network access controls
- Testing connectivity between services using curl
- TLS handshake and certificate validation using OpenSSL
- Debugging connectivity issues between public and private services

4. Tools Used

- AWS EC2, VPC, IAM
- AWS CLI
- curl
- OpenSSL
- Linux Networking tools

5. Key Findings

- Proper subnet isolation is critical for securing backend services
- TLS misconfigurations can disrupt or weaken secure communication
- Internal services must not be directly exposed to public networks
- Debugging tools like curl and OpenSSL are essential for troubleshooting

6. Security Improvements

- Restrict backend access to internal traffic only
- Use trusted certificates and enforce validation
- Apply strict security group rules
- Monitor internal traffic patterns

7. Conclusion

This project reinforced the importance of secure architecture design in cloud environments, particularly in controlling communication between public and private services ,network segmentation importance, and basic cloud communication.